

UML Metamodel

Formálny základ jazyka Unified Modeling Language

SOFTVÉROVÁ ARCHITEKTÚRA



Cieľové skupiny

IKT architekti, analytici, vývojové tímy

1. Biznis analytici modelujúci procesy pomocou UML diagramov
2. Systémoví analytici navrhujúci štruktúru aplikácií a dátových modelov
3. Softvéroví architekti definujúci triedy, komponenty a ich vzťahy
4. Vývojári pracujúci s model-driven prístupmi (MDE, MDA)
5. Tímy používajúce UML nástroje (Enterprise Architect)
6. Architekti navrhujúci viacvrstvové a komponentové systémy
7. Špecialisti implementujúci transformácie modelov na kód
8. Tímy pracujúce s OCL pri validácii modelov
9. IT oddelenia zavádzajúce štandardy modelovania a dokumentácie
10. Organizácie formalizujúce návrh systémov pomocou UML metamodelu

Manažéri, konzultanti, rozhodovatelia

1. IT manažéri riadiaci kvalitu návrhu systémov a architektúr
2. Enterprise architekti nastavujúci modelovacie štandardy organizácie
3. Konzultanti hodnotiaci konzistenciu a správnosť modelov
4. CIO a CTO definujúci metodiky modelovania a dokumentácie
5. Projektoví manažéri koordinujúci návrh komplexných IT riešení
6. Organizácie implementujúce model-driven engineering
7. Firmy optimalizujúce vývoj pomocou modelovania a automatizácie
8. Spoločnosti vyžadujúce štandardizovanú architektonickú dokumentáciu
9. Tímy pripravujúce sa na škálovanie systémov a architektúr
10. Rozhodovatelia požadujúci presné a validovateľné modely systémov



Čo sa naučíme?

1. Čo je UML metamodel?
2. Aký je rozdiel medzi UML modelom a UML metamodelom?
3. Čo predstavujú vrstvy M0 až M3 v UML?
4. Čo je MOF (Meta Object Facility) a aký má vzťah k UML?
5. Čo je metatrieda v UML metamodeli?
6. Ako je v metamodeli definovaná trieda (Class)?
7. Ako metamodel reprezentuje vzťahy medzi prvkami?
8. Čo je Property v UML metamodeli?
9. Akú úlohu má OCL v UML metamodeli?
10. Prečo je UML metamodel dôležitý pre nástroje a automatizáciu?

Čo je UML?

Štandardizovaný jazyk

- UML (Unified Modeling Language) je vizuálny modelovací jazyk
- Definovaný organizáciou **OMG (Object Management Group)**
- Nejde len o sadu diagramov
- Je to formálne špecifikovaný jazyk s presne definovanou syntaxou a sémantikou

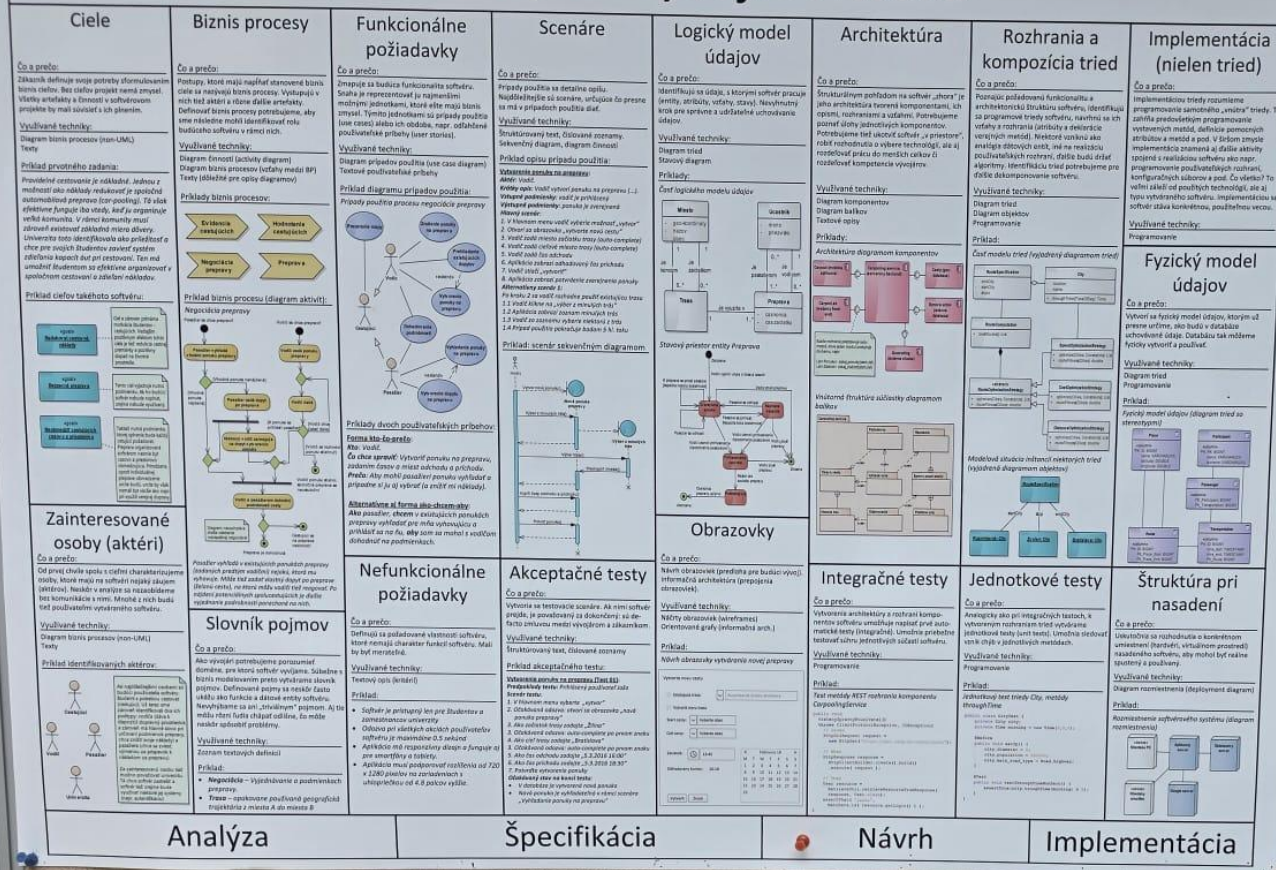
Prečo metamodel?

Bez metamodelu by UML bolo iba neformálnou kresliacou notáciou.

Metamodel zabezpečuje:

1. Konzistenciu a jednoznačnosť modelov
2. Interpretovateľnosť nástrojmi
3. Automatizované spracovanie a validáciu
4. Možnosť transformácií modelov

Artefakty vo vývoji softvéru



Nová kancelária InQb na FEI STU !

Potrebuje odbornú pomoc pri založení firmy a rozbehnutí podnikania?

Na FEI STU bola zriadená pobočka Univerzitného technologického inkubátora InQb STU v rámci projektu Univerzitného inkubátora Science City STU.

Kontaktná kancelária sa nachádza v Bloku D v chodbe za výťahmi – miestnosť DP01

Individuálnu konzultáciu si môžete dohodnúť na kontaktoch:

info@inqb.sk, 0918 669 209
www.inqb.sk

Artefakty vo vývoji softvéru

Ciele

Čo a prečo:

Zákazník definuje svoje potreby sformulovaním biznis cieľov. Bez cieľov projekt nemá zmysel. Všetky artefakty a činnosti v softvérovom projekte by mali súvisieť s ich plnením.

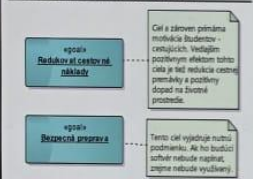
Využívané techniky:

Diagram biznis procesov (non-UML)
Texty

Priklad prvotného zadania:

Pravidelné cestovanie je nákladné. Jednou z možností ako náklady redukovať je spoločná automobilová preprava (car-pooling). Tá však efektívne funguje iba vtedy, keď ju organizuje veľká komunita. V rámci komunity musí zároveň existovať základná miera dôvery. Univerzita toto identifikovala ako príležitosť a chce pre svojich študentov zaviesť systém zdieľania kapacít aut pri cestovaní. Ten má umožniť študentom sa efektívne organizovať v spoločnom cestovaní a zdieľaní nákladov.

Priklad cieľov takéhoto softvéru:



Zainteresované osoby (aktéri)

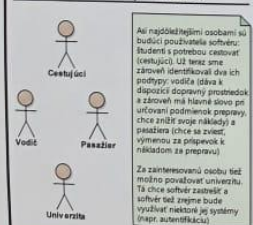
Čo a prečo:

Od prvej chvíľky spolu s cieľmi charakterizujeme osoby, ktoré majú na softvéri nejaký záujem (aktéri). Neskôr v analýze sa zaoberáme bez komunikácie s nimi. Mnohé z nich budú tiež používateľmi vytváraného softvéru.

Využívané techniky:

Diagram biznis procesov (non-UML)
Texty

Priklad identifikovaných aktérov:



Biznis procesy

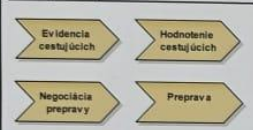
Čo a prečo:

Postupy, ktoré majú naplňovať stanovené biznis ciele sa nazývajú biznis procesy. Vystupujú v nich tiež aktéri a rôzne ďalšie artefakty. Definovať biznis procesy potrebujeme, aby sme následne mohli identifikovať rolu budúceho softvéru v rámci nich.

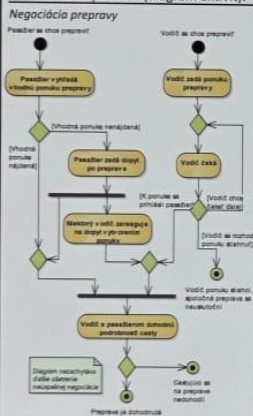
Využívané techniky:

Diagram činnosti (activity diagram)
Diagram biznis procesov (vzťahy medzi BP)
Texty (dôležité pre opisy diagramov)

Priklady biznis procesov:



Priklad biznis procesu (diagram aktivít):



Pasážier vyhľadá v existujúcich ponukách prepravy (zadaných predtým vodičmi) nejakú, ktorá mu vyhovuje. Má tiež zvoliť vlastný dopravný prostried (číslo auta), na ktorom má byť vodič tiež reagoval. Po nájdenej potenciálnej spolupráci je ďalšie vyjednanie podrobností ponechané na nich.

Slovník pojmov

Čo a prečo:

Ako vývojári potrebujeme porozumieť doméne, pre ktorú softvér vyvíjame. Súbežne s biznis modelovaním preto vytvárame slovník pojmov. Definované pojmy sa neskôr často ukážu ako funkcie a dátové entity softvéru. Nevýhoda sa ani „triviálnym“ pojmom. Aj tie môžu rôzni ľudia chápať odlišne, čo môže viesť k problémom.

Využívané techniky:

Zoznam textových definícií

- Priklad:**
- Negociácia – Vyjednanie o podmienkach prepravy.
 - Nová ponuka je vyhľadanie v rámci scenára „Vyhľadanie ponuky na prepravu“.

Funkcionálne požiadavky

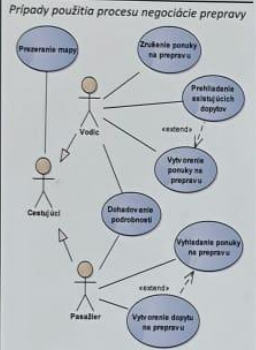
Čo a prečo:

Zmapuje sa budúca funkcionálna softvéru. Snaha je reprezentovať ju najmenšími možnými jednotkami, ktoré ešte majú biznis zmysel. Týmto jednotkami sú prípady použitia (use cases) alebo ich obdoba, napr. odľahčené používateľské príbehy (user stories).

Využívané techniky:

Diagram prípadov použitia (use case diagram)
Textové používateľské príbehy

Priklad diagramu prípadov použitia:



Priklady dvoch používateľských príbehov:

Forma kto-čo-prečo:
Kto: Vodič.
Čo: chce spraviť: Vytvoriť ponuku na prepravu, zadaním čísla a miesta odchodu a príchoďu.
Prečo: Aby mohol, pasážieri ponuku vyhľadajú a prípadne si ju aj vybrať (a znížiť mi náklady).

Alternatívne aj forma ako-čím-aby:
Ako: pasážier, **čím:** v existujúcich ponukách prepravy vyhľadajú pre mňa vyhovujúcu a príhlásiť sa na ňu, **aby som sa mohol s vodičom dohodnúť na podmienkach.**

Nefunkcionálne požiadavky

Čo a prečo:

Definujú sa požadované vlastnosti softvéru, ktoré nemajú charakter funkcií softvéru. Mali by byť merateľné.

Využívané techniky:

Textový opis (kritériá)

Priklad:

- Softvér je prístupný len pre študentov a zamestnancov univerzity.
- Odpoveď na všetky akcie používateľov softvéru je maximálne 0.5 sekundy.
- Aplikácia má responzívny dizajn a funguje aj pre smartfóny a tablety.
- Aplikácia musí podporovať rozlíšenia od 720 x 1280 pixelov na zariadeniach s uhlopriečkou od 4.8 palcov vyššie.

Scenáre

Čo a prečo:

Prípady použitia sa detailne opíšu. Najdôležitejšie sú scenáre, určujúce čo presne sa má v prípadoch použitia diať.

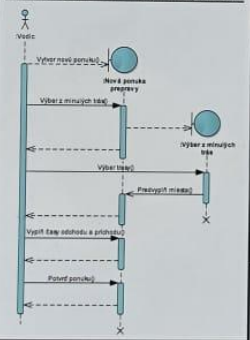
Využívané techniky:

Štruktúrovaný text, číslované zoznamy.
Sekvenčný diagram, diagram činnosti

Priklad opisu prípadu použitia:

Vytvorenie ponuky na prepravu:
Akteři: Vodič.
Kritériá opisu: Vodič vytvorí ponuku na prepravu (...).
Vstupné podmienky: vodič je prihlásený.
Výstupné podmienky: ponuka je zverejnená.
Hlavný scenár:
1. V hlavnom menu vodič vyberie možnosť „vytvor“
2. Otvorí sa obrazovka „vytvor novú cestu“
3. Vodič zadá miesto začiatku trasy (auto-complete)
4. Vodič zadá cieľové miesto trasy (auto-complete)
5. Vodič zadá čas odchodu
6. Aplikácia zobrazí odhadovaný čas príchodu
7. Vodič stlačí „vytvor“
8. Aplikácia zobrazí potvrdenie zverejnenia ponuky.
Alternatívny scenár 1:
Po kroku 2 sa vodič rozhodne použiť existujúcu trasu.
1.1 Vodič klikne na „vyber z minulých trás“
1.2 Aplikácia zobrazí zoznam minulých trás
1.3 Vodič po zozname vyberie niektorú z trás
1.4 Prípade použitia pokračuje bodom 5 hl. toku

Priklad: scenár sekvenčným diagramom



Akceptačné testy

Čo a prečo:

Vytvoria sa testovacie scenáre. Ak nimi softvér prejde, je považovaný za dokončený: sú de facto zmluvou medzi vývojárom a zákazníkom.

Využívané techniky:

Štruktúrovaný text, číslované zoznamy

Priklad akceptačného testu:

Vytvorenie ponuky na prepravu [Test 01]:
Predpoklady testu: Prihlásený používateľ / so scenár testu:
1. V hlavnom menu vyberie „vytvor“
2. Očakávaná odpoveď: otvorí sa obrazovka „nová ponuka prepravy“
3. Ako začiatok trasy zadá „Žilina“
4. Očakávaná odpoveď: auto-complete po prvom znaku
5. Ako čas odchodu zadá „5.3.2016 16:00“
6. Ako čas príchodu zadá „5.3.2016 18:30“
7. Potvrdí vytvorenie ponuky
Očakávaný stav na konci testu:
• V databáze je vytvorená nová ponuka
• Nová ponuka je vyhľadateľná v rámci scenára „Vyhľadanie ponuky na prepravu“

Logický model údajov

Čo a prečo:

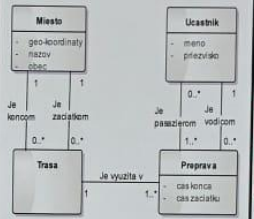
Identifikujú sa údaje, s ktorými softvér pracuje (entty, atribúty, vzťahy, stavy). Nevýhoda je, že pre správne a udržateľné uchovávanie údajov.

Využívané techniky:

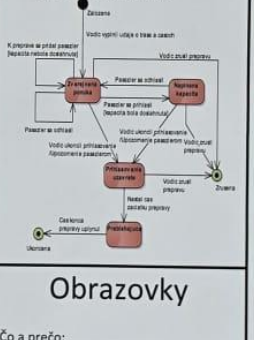
Diagram tried
Stavový diagram

Priklady:

Časť logického modelu údajov



Stavový priestor entity Preprava



Obrazovky

Čo a prečo:

Návrh obrazoviek (predloha pre budúci vývoj). Informačná architektúra (počítanie obrazoviek).

Využívané techniky:

Náčrty obrazoviek (wireframes)
Orientované grafy (informačná arch.)

Priklad:

Návrh obrazovky vytvárania novej prepravy



Architektúra

Čo a prečo:

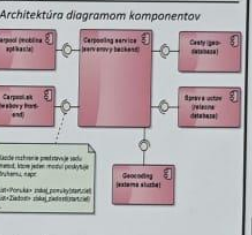
Štruktúrnym pohľadom na softvér „zhora“ je jeho architektúra tvorená komponentami, ich opismi, rozhraniami a vzťahmi. Potrebujeme poznať úlohy jednotlivých komponentov. Potrebujeme tiež ukotviť softvér „v priestore“, robí rozhodnutia o výbere technológií, ale aj rozdeľovať prácu do menších celkov či rozdeľovať kompetencie vývojárov.

Využívané techniky:

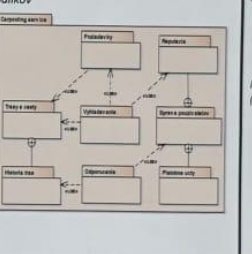
Diagram komponentov
Diagram balíkov
Textové opisy

Priklady:

Architektúra diagramu komponentov



Vnútroštruktúra súčastí diagramu balíkov



Rozhrania a kompozícia tried

Čo a prečo:

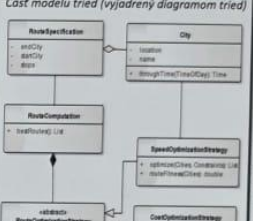
Poznájú požadovanú funkcionálnu a architektonickú štruktúru softvéru, identifikujú sa programové triedy softvéru, navrhujú sa ich vzťahy a rozhrania (atribúty a deklarácie verejných metód). Niektoré vzniknú ako analógia dátových entít, iné na realizáciu používateľských rozhraní, ďalšie budú držať algoritmy. Identifikáciu tried potrebujeme pre ďalšie dekomponovanie softvéru.

Využívané techniky:

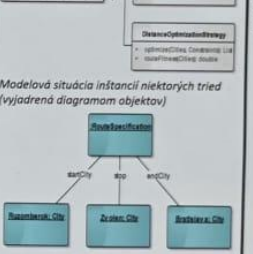
Diagram tried
Diagram objektov
Programovanie

Priklady:

Časť modelu tried (vyjadrený diagramom tried)



Modelová situácia inštancií niektorých tried (vyjadrená diagramom objektov)



Jednotkové testy

Čo a prečo:

Analogicky ako pri integračných testoch, k vytváreniu rozhraniam tried vytvárame jednotkové testy (unit tests). Umožnia sledovať vznik chýb v jednotlivých metódach.

Využívané techniky:

Programovanie

Priklad:

Test metódy REST rozhrania komponentu CarpoolingService

```
public void testCarpoolingService() {  
    CarpoolingService service = new CarpoolingService();  
    // ...  
    // ...  
    // ...  
}
```

Implementácia (nielen tried)

Čo a prečo:

Implementáciou triedy rozumieme programovanie samotného „vnútra“ triedy. To zahŕňa predovšetkým programovanie vystavených metód, definície pomocných atribútov a metód a pod. V širšom zmysle implementácia znamená aj ďalšie aktivity spojené s realizáciou softvéru ako napr. programovanie používateľských rozhraní, konfiguračných súborov a pod. Čo všetko? To veľmi záleží od použitých technológií, ale aj typu vytváraného softvéru. Implementáciou sa softvér stáva konkrétnou, použiteľnou vecou.

Využívané techniky:

Programovanie

Fyzický model údajov

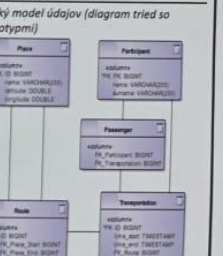
Čo a prečo:

Vytvorí sa fyzický model údajov, ktorým už presne určíme, ako budú v databáze uchovávané údaje. Databázu tak môžeme fyzicky vytvoriť a používať.

Využívané techniky:

Diagram tried
Programovanie

Priklad:



Štruktúra pri nasadení

Čo a prečo:

Uskutočnia sa rozhodnutia o konkrétnom umiestnení (hardvéri, virtuálnom prostredí) nasadeného softvéru, aby mohol byť reálne spustený a používaný.

Využívané techniky:

Diagram rozmiestnenia (deployment diagram)

Priklad:



Analýza

Špecifikácia

Návrh

Implementácia

ABOUT THE UNIFIED MODELING LANGUAGE SPECIFICATION VERSION 2.5.1

2.5.1 • UML • SPECIFICATIONS

UML® — Unified Modeling Language

A specification defining a graphical language for visualizing, specifying, constructing, and documenting the artifacts of distributed object systems.



Title: Unified Modeling Language

Acronym: UML®

Version: 2.5.1

Document Status: formal ⓘ

Publication Date: December 2017

Categories: [Modeling](#) [Software Engineering](#) [Platform](#)

IPR Mode ⓘ RF-Limited ⓘ

TABLE OF CONTENTS

- [About the Specification](#)
- [Issues associated with this specification](#)
- [Specification Documents](#)
 - [Normative Documents](#)
 - [Normative Machine Readable Documents](#)
 - [Informative Documents](#)
- [History](#)
 - [Formal Versions](#)

ISSUES ASSOCIATED WITH THIS SPECIFICATION



Issues Reported in this Specification - Version 2.5.1 only



Issues Fixed in this Specification - Version 2.5.1 only



Report an issue



Issue Tracker (Members Only)

SPECIFICATION DOCUMENTS

NORMATIVE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
Specification	PDF	UML/2.5.1/PDF	formal/17-12-05

NORMATIVE MACHINE READABLE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
UML 2.5.1 Abstract Syntax Metamodel	XMI	UML/20161101/UML.xmi	ptc/18-01-01
UML 2.5.1 Primitive Types	XMI	UML/20161101/PrimitiveTypes.xmi	ptc/18-01-02
UML 2.5.1 Standard Profile	XMI	UML/20161101/StandardProfile.xmi	ptc/18-01-03
UML 2.5.1 Diagram Interchange Metamodel	XMI	UML/20161101/UMLDI.xmi	ptc/18-01-04

INFORMATIVE DOCUMENTS



ABOUT THE UNIFIED MODELING LANGUAGE SPECIFICATION VERSION 2.4.1

2.4.1 • UML • SPECIFICATIONS

UML® — Unified Modeling Language

This specification defines the Unified Modeling Language (UML), revision 2. The objective of UML is to provide system architects, software engineers, and software developers with tools for analysis, design, and implementation of softwarebased systems as well as for modeling business and similar processes.

Title: Unified Modeling Language

Acronym: UML®

Version: 2.4.1

This version was superseded by a newer inventory. The latest version can be found here: [UML](#)

Document Status: formal ⓘ

Publication Date: júla 2011

Categories:

Modeling

Software Engineering

Platform

IPR Mode ⓘ RF-Limited ⓘ



TABLE OF CONTENTS

- [About the Specification](#)
- [Companies that have contributed to the development of this Specification](#)
- [Issues associated with this specification](#)
- [Specification Documents](#)
 - [Normative Documents](#)
 - [Normative Modeling Request Documents](#)



ITU-T work programme

YOU ARE HERE HOME > ITU-T WORK PROGRAMME

SHARE

Home : [ITU-T Home](#) : [ITU-T Work Programme](#) : [M.3020](#)

ITU-T A.5 justification information for referenced document [OMG UML Version 2.4.1](#) in draft [M.3020](#)

1. **Clear description of the referenced document:**

Name:[OMG UML Version 2.4.1](#)

Title:OMG. OMG Unified Modeling Language (OMG UML), Superstructure. Version 2.4.1, document no. formal/2011 08 06.

2. **Status of approval:**

Approved

3. **Justification for the specific reference:**

This Recommendation refers to [OMG UML 2.4.1](#) on UML basic model elements, class, types, and semantics of the above mentioned items.

4. **Current information, if any, about IPR issues:**

The attention of adopters is directed to the possibility that compliance with or adoption of [OMG specifications](#) may require use of an invention covered by patent rights. [OMG](#) shall not be responsible for identifying patents for which a license may be required by any [OMG specification](#), or for conducting legal inquiries into the legal validity or scope of those patents that are brought to its attention. [OMG specifications](#) are prospective and advisory only. Prospective users are responsible for protecting themselves against liability for infringement of patents.

5. **Other useful information describing the "Quality" of the document:**

Beginning with [UML 2.0](#), the UML Specification was split into two complementary specifications: Infrastructure and Superstructure. The UML infrastructure specification defines the foundational language constructs required for [UML 2.4.1](#). It is complemented by UML Superstructure, which defines the user level constructs required for [UML 2.4.1](#). The two complementary specifications constituted a complete specification for the UML 2 modeling language. All [OMG specifications](#) are subject to continuous review and improvement. [UML 2.5](#) was a minor revision rewrite that included the [UML 2.4.1 Infrastructure](#) and Superstructure in a single document and is considered technically equivalent to [UML 2.4.1](#).

6. **The degree of stability or maturity of the document:**

Version 2.4.1 Superstructure is stable, and has been absorbed into [UML version2.5](#). Version 2.5 is formally a minor revision to the [UML 2.4.1 specification](#), having been substantially rewritten as solicited by the UML Specification Simplification RFP ad/09-12-10. It supersedes [formal/2011-08-05 \(Infrastructure\)](#) and [formal/2011-08-06 \(Superstructure\)](#). Version 2.5 has been stable since 2015.

SPECIFICATION DOCUMENTS

NORMATIVE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
Specification - Infrastructure	PDF	UML/2.4.1/Infrastructure/PDF	formal/11-08-05
Specification - Superstructure	PDF	UML/2.4.1/Superstructure/PDF	formal/11-08-06

NORMATIVE MACHINE READABLE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
This file contains the XMI of the UML Infrastructure v2.4 as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/Infrastructure.xmi	ptc/10-11-17
This file contains the XMI of the UML Superstructure v2.4 as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/Superstructure.xmi	ptc/10-11-18
This file contains the XMI of the UML L0 v2.4 compliance model as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/L0.xmi	ptc/10-11-19
This file contains the XMI of the UML L1 v2.4 compliance model as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/L1.xmi	ptc/10-11-20
This file contains the XMI of the UML L2 v2.4 compliance model as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/L2.xmi	ptc/10-11-21
This file contains the XMI of the UML L3 v2.4 compliance model as an instance of MOF 2.4 using XMI 2.4.	XMI	UML/20100901/L3.xmi	ptc/10-11-22

Čo je **Metamodel**?

Metamodel je **model modelov** t.j. abstraktná vrstva, ktorá opisuje štruktúru, pravidlá a význam modelovacieho jazyka.

Model

Reprezentuje konkrétny systém (napr. diagram tried aplikácie)

Metamodel

Reprezentuje samotný jazyk UML a definuje metatriedy ako Class, Property, Operation

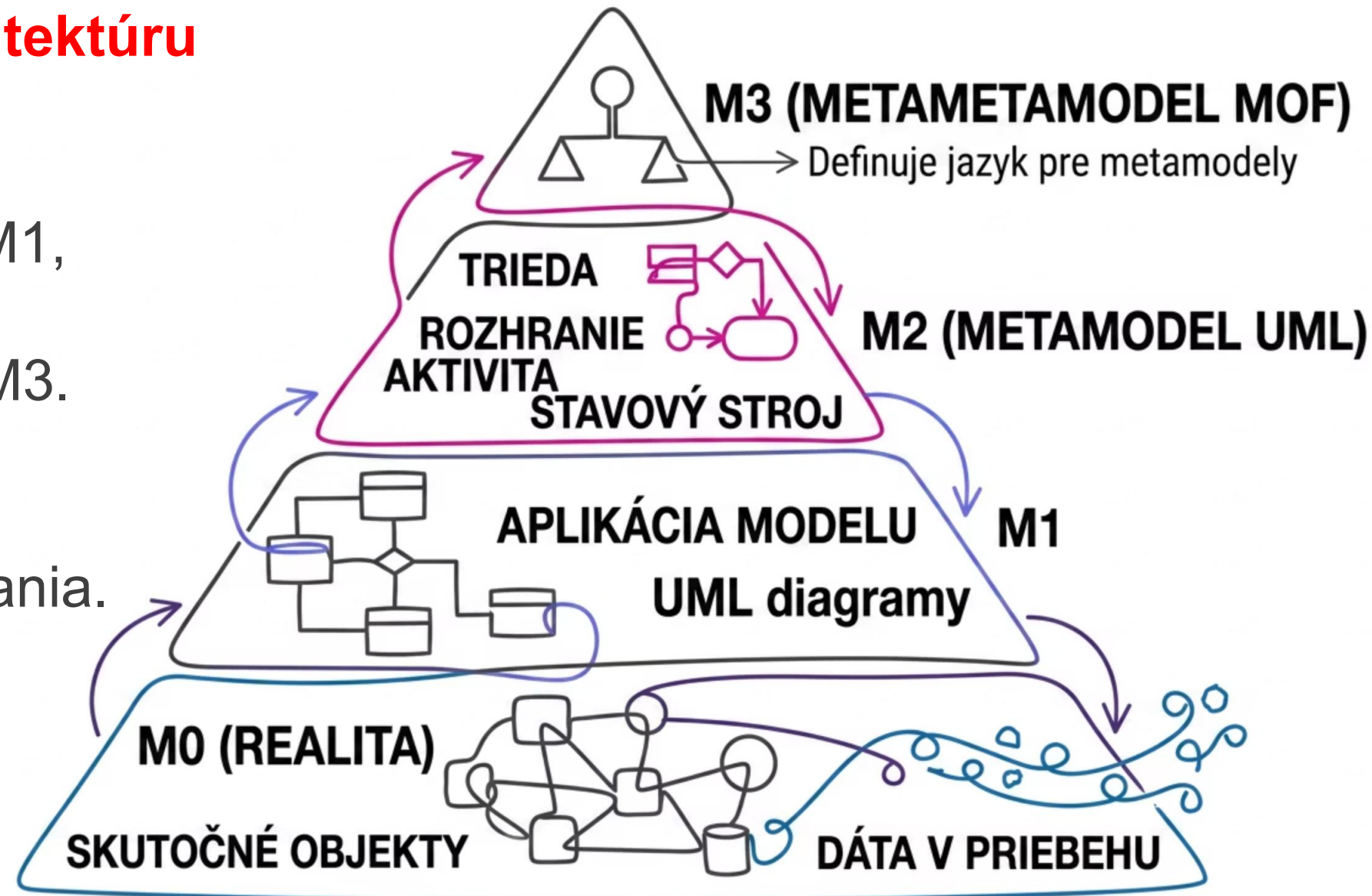
Kľúčová vlastnosť

Každý prvok UML diagramu musí byť **inštanciou nejakej metatriedy metamodelu**



UML Architektúra M0 – M3

- OMG definuje **štvorvrstvovú architektúru** ako základ modelovania v UML.
- Objekty M0 sú inštanciami prvkov M1, prvky M1 sú inštanciami M2 a prvky M2 sú definované pomocou M3.
- Táto architektúra umožňuje presnú formalizáciu a automatizáciu modelovania.





OMG Issue

UMLR — Need packages overview diagram and explicit depiction of package dependencies

Key: [UMLR-472](#) · OMG Task Force: [UML 2.6 RTF](#)

Open

Closed

All

Key: UMLR-472

Legacy Issue Number: 18195

Status: **OPEN**

Source: Simula Research Laboratory ([Dr. Bran Selic](#))

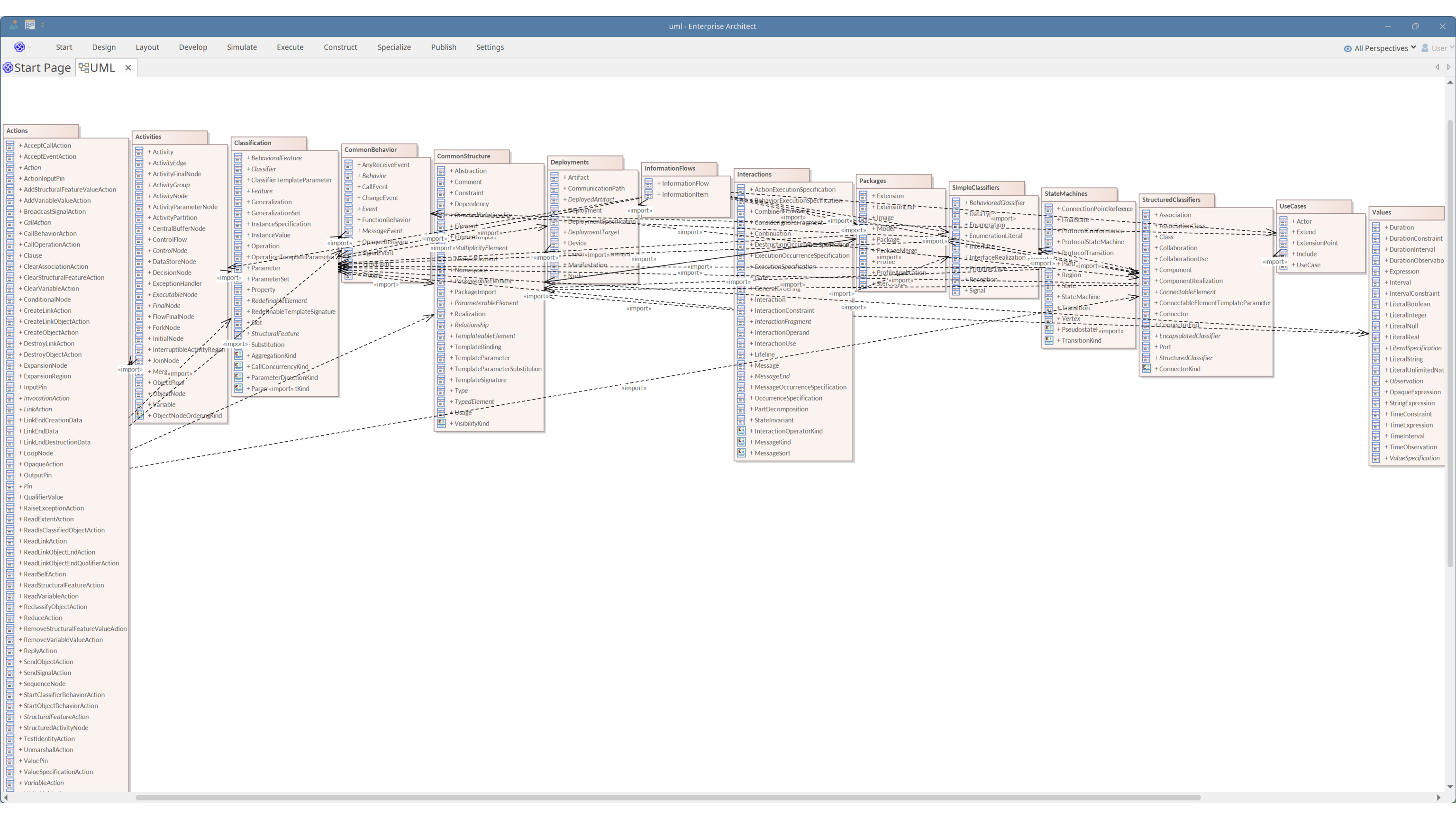
Summary: In the current 2.5 metamodel, there is a diagram that simply shows all the various packages that comprise it. Note, however, that we do not have such a diagram in the spec itself. In fact, as far as I can tell, it seems that the spec does not even mention explicitly that the metamodel is divided up into these packages.

Furthermore, it is useful to see the dependencies between the various packages captured explicitly in diagrams. In a sense, this shows the intended couplings between the various modules, which is usually important architectural information. Not showing them explicitly either in the spec or in a metamodel diagram is obscuring this and could lead to corruption of the architecture in subsequent maintenance (e.g., the introduction of undesired and even incorrect couplings between the packages).

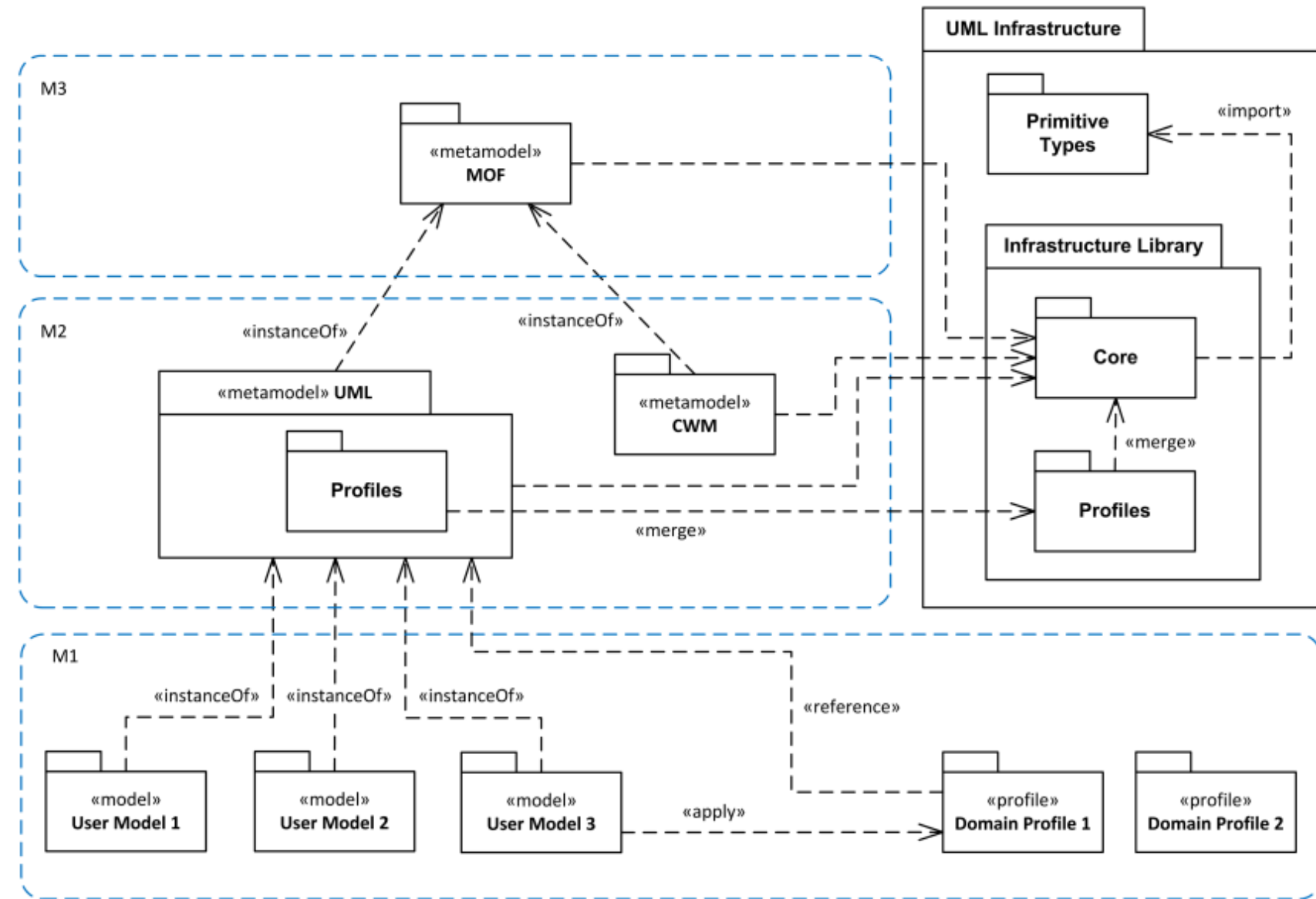
Reported: [UML 2.5b1](#) — Tue, 23 Oct 2012 04:00 GMT

Updated: Fri, 6 Mar 2015 20:57 GMT



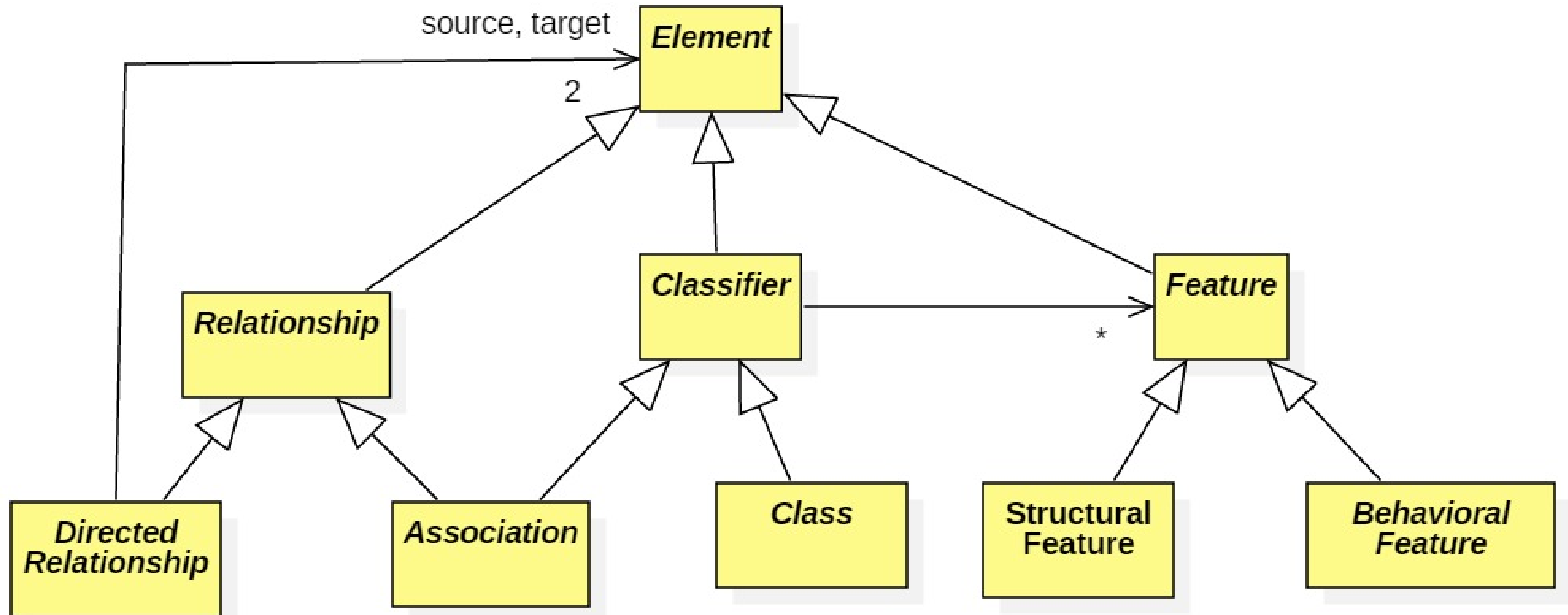


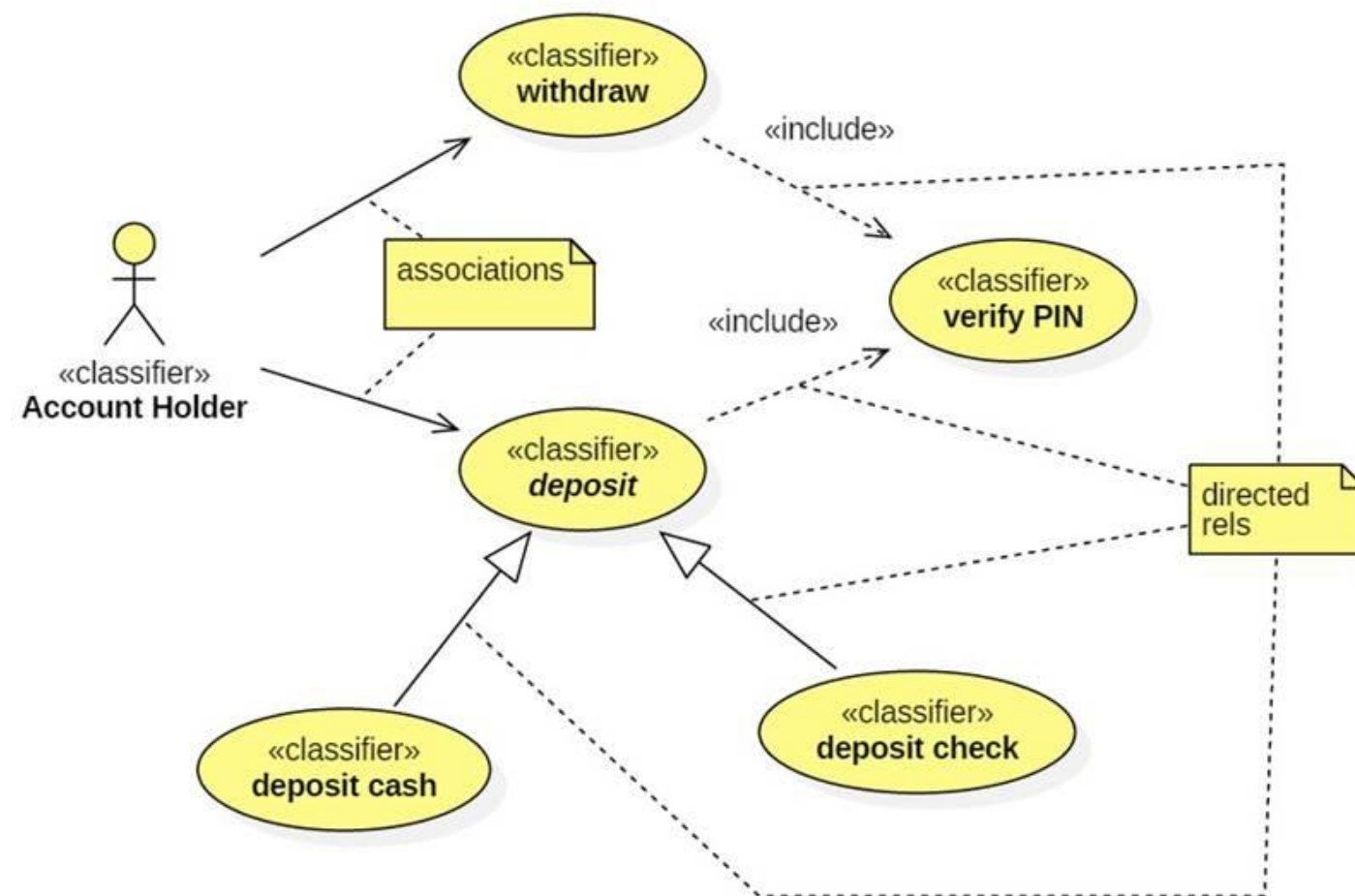
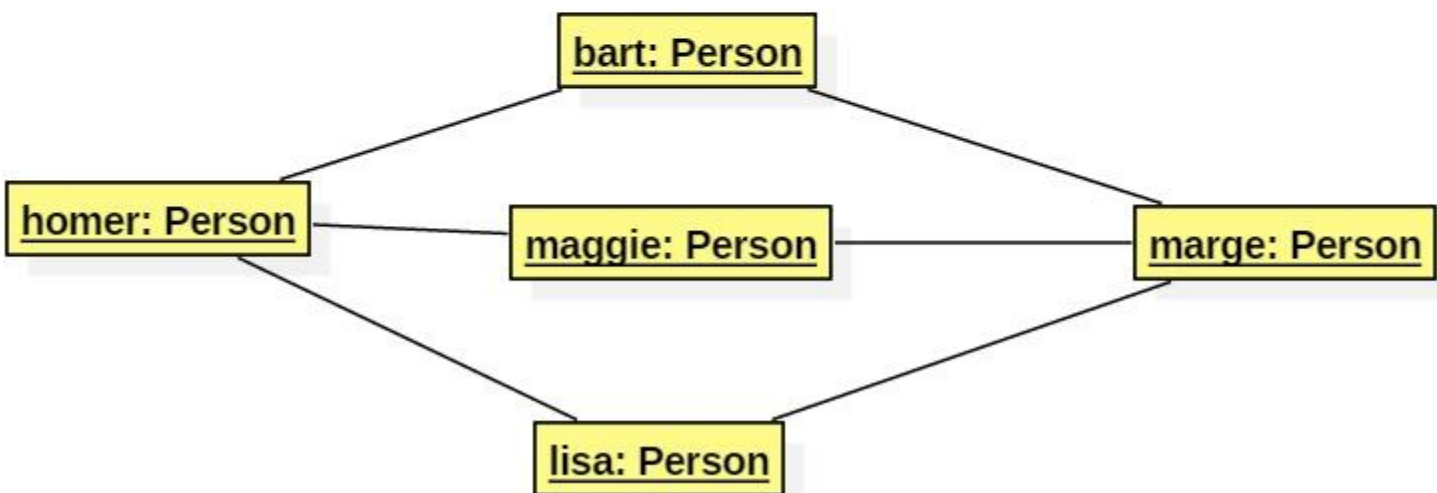
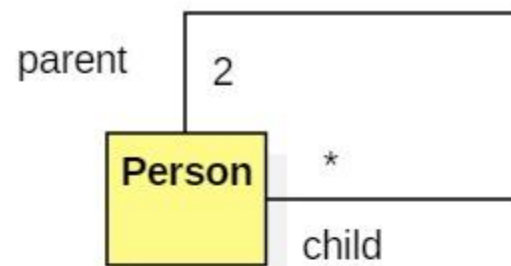
UML Metamodel



*The meta-metamodeling layer forms the foundation of the metamodeling hierarchy. The primary responsibility of this layer is to define the language for specifying a metamodel. The layer is often referred to as **M3**, and **MOF** is an example of a meta-metamodel.*

UML Metamodel





The screenshot displays the UML modeling tool interface. On the left, the 'Model' tree shows the 'Imported UML 2.5.1.' package structure, which includes sub-packages like 'Actions', 'Activities', 'Classification', 'CommonBehavior', 'CommonStructure', 'Deployments', 'InformationFlows', 'Interactions', 'Packages', 'SimpleClassifiers', 'StateMachines', 'StructuredClassifiers', 'UseCases', and 'Values'. The 'UML' package is highlighted. In the center, the 'Import' dialog box is open, showing the 'Merge Options' section with a 'Merge XML' button and a 'Baseline Package after Import' checkbox. The 'Progress' bar is visible. On the right, the 'General' tab of the 'Properties' window shows the package details, including 'Name: UML', 'Type: Package', 'Stereotype', 'Alias', 'Keywords', 'Status: Proposed', 'Version: 1.0', 'URI', 'Visibility: Public', and 'Project'. The 'Notes' window is also visible at the bottom right.

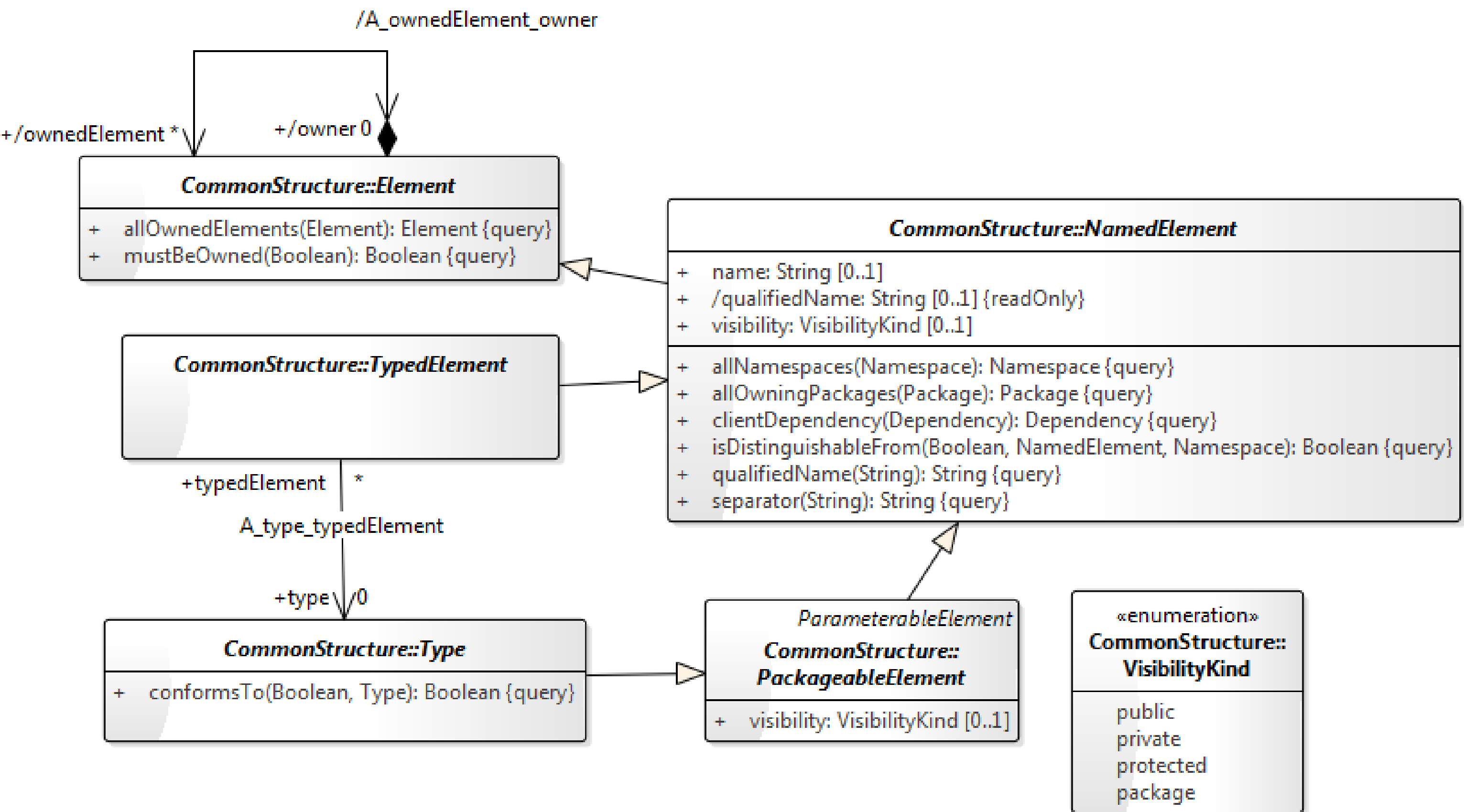
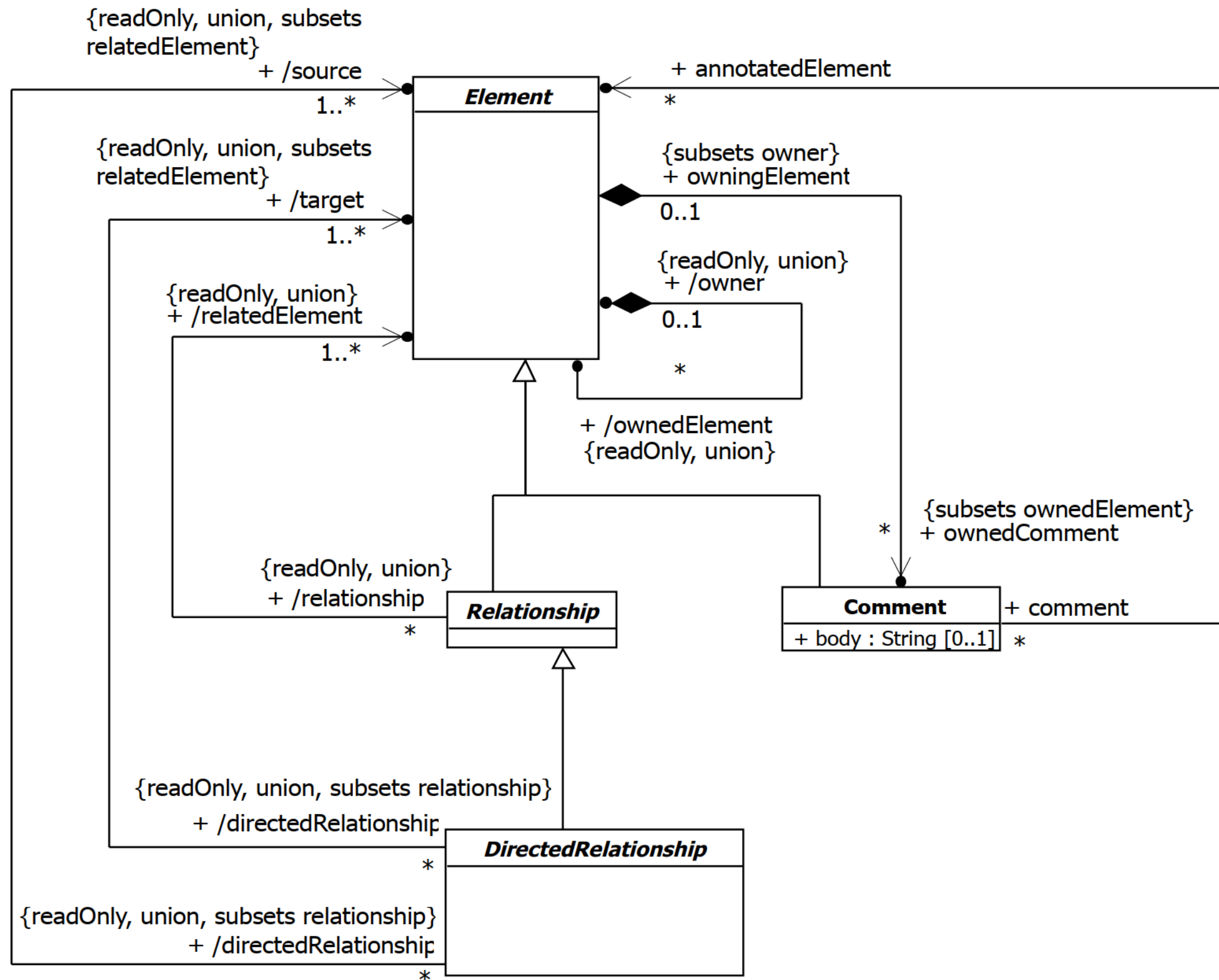


Diagram metamodelu UML

7.1 Root kapitole 7.2.2 Abstract Syntax



File

Home

Convert

Edit

Fill & Sign

Organize

Comment

View

Form

Protect

Share

Connect

Accessibility

Help

Tell me what you want to do...

Start

Unified Modeling Langu...

1 Scope

2 Conformance

3 Normative References

4 Terms and Definitions

5 Notational Conventions

6 Additional Information

7 Common Structure

8 Values

9 Classification

10 Simple Classifiers

11 Structured Classifiers

12 Packages

13 Common Behavior

14 StateMachines

15 Activities

16 Actions

17 Interactions

18 UseCases

19 Deployments

20 InformationFlows

21 Primitive Types

22 Standard Profile

Annex A: Diagrams

Annex B: UML Diagram Interchange

Annex C: Keywords

Annex D: Tabular Notation for Sequence Diagrams

Annex E: XMI Serialization and Schema

Unified Modeling Language, v2.5.1 - Foxit PhantomPDF

Find

7.2

Root

7.2.1

Summary

7.2.2

Abstract Syntax

The root concepts of Element and Relationship provide the basis for all other modeling concepts in UML.

```
classDiagram
    class Element {
        {readOnly, union, subsets relatedElement} + /source 1..*
        {readOnly, union, subsets relatedElement} + /target 1..*
        {readOnly, union} + /relatedElement 1..*
        + annotatedElement *
        {subsets owner} + owningElement 0..1
        {readOnly, union} + /owner 0..1
        + /ownedElement {readOnly, union} *
    }
    class Relationship {
        {readOnly, union} + /relationship *
    }
    class Comment {
        {subsets ownedElement} + ownedComment *
        + comment
        + body : String [0..1] *
    }
    class DirectedRelationship {
        {readOnly, union, subsets relationship} + /directedRelationship *
        {readOnly, union, subsets relationship} + /directedRelationship *
    }
    Element <|-- Relationship
    Relationship <|-- Comment
    Relationship <|-- DirectedRelationship
    Element --> Element : {readOnly, union, subsets relatedElement} + /source 1..*
    Element --> Element : {readOnly, union, subsets relatedElement} + /target 1..*
    Element --> Element : {readOnly, union} + /relatedElement 1..*
    Element --> Element : + annotatedElement *
    Element --> Element : {subsets owner} + owningElement 0..1
    Element --> Element : {readOnly, union} + /owner 0..1
    Element --> Element : + /ownedElement {readOnly, union} *
    Relationship --> Relationship : {readOnly, union} + /relationship *
```

Figure 7.1

Root

63 / 796

150%

MOF - Meta Object Facility

MOF je štandard OMG, ktorý slúži ako **meta-metamodel** pre UML a iné modelovacie jazyky. **UML metamodel** je **implementovaný ako MOF model**.

Class

Základná konštrukcia
pre definíciu tried

Property

Vlastnosti a atribúty prvkov

Association

Vzťahy medzi prvkami

Package

Zoskupovanie prvkov

Constraint

Obmedzenia a pravidlá

 MOF umožňuje vytvárať vlastné modelovacie jazyky - DSL (Domain Specific Languages)

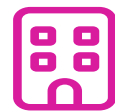
Štruktúra UML metamodelu

UML metamodel je rozsiahly a modulárny, organizovaný do **balíkov (packages)**, ktoré logicky zoskupujú súvisiace prvky.



Infrastructure

Základné abstrakcie -
balík Kernel s
metatriedami Element,
NamedElement,
Package



Superstructure

Konkrétne UML
konštrukcie - Classes,
Activities, Interactions,
Components



CommonStructure

Všeobecné koncepty
ako Dependency a
Constraint, zdieľané
naprieč balíkmi

Základné metatriedy UML

Element

Koreňová metatrieda - každý prvok UML dedí z tejto triedy.

NamedElement & Namespace

Rozširuje Element o názov;
Namespace umožňuje zoskupovanie prvkov do menného priestoru.

Type & Class

Type je abstrakcia pre typy; Class reprezentuje triedu s atribútmi (Property) a operáciami (Operation).

Association & Constraint

Association definuje vzťah medzi dvoma alebo viacerými triedami;
Constraint reprezentuje obmedzenia nad modelom.

Vzt'ahy v metamodeli



Generalization

Dedičnosť medzi triedami



Association

Štruktúrálny vzťah medzi triedami



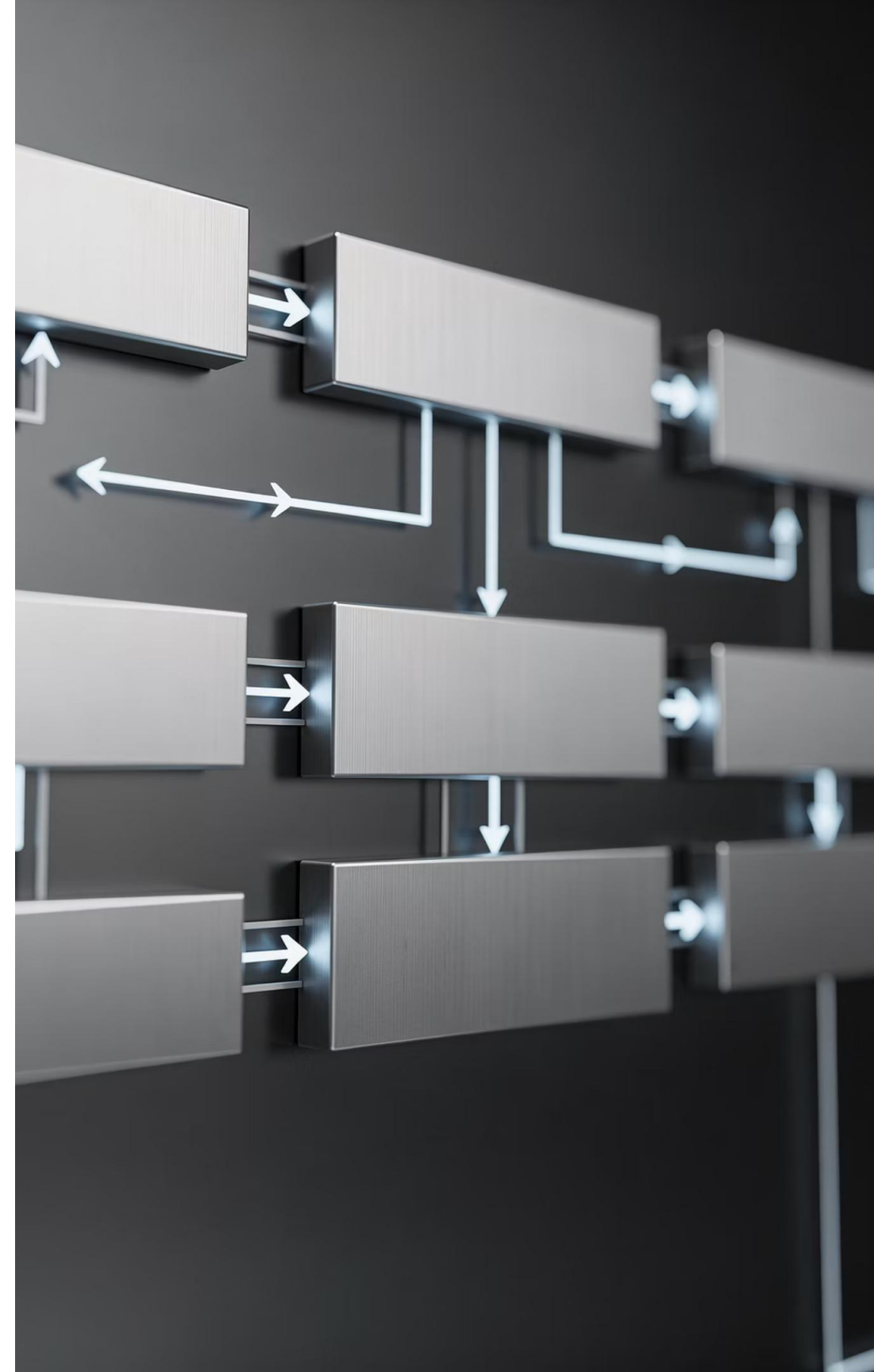
Dependency

Slabší vzťah závislosti



Aggregation & Composition

Špeciálne formy asociácie s presne určenou sémantikou



OCL - Object Constraint Language

Čo je OCL?

- Je formálny jazyk na definovanie obmedzení v UML modeloch.
- Zabezpečuje, že model spĺňa pravidlá, ktoré nie je možné vyjadriť len graficky.

Používa sa na:

1. definovanie invariantov
2. validáciu modelov
3. špecifikáciu pravidiel

Príklady OCL výrazu

```
context Person  
inv: self.age > 0
```

Tento výraz zabezpečuje, že model nemôže obsahovať osobu s neplatným vekom.

```
context Account  
inv: self.balance >= 0
```

Tento výraz zabezpečuje, že zostatok na účte nemôže byť záporný.

OCL - Object Constraint Language

pre – podmienka pred vykonaním operácie

- **Príklad predpodmienka**

```
context Account::withdraw(amount: Real)
pre: amount > 0 and amount <= self.balance
```

- **Popis:**

Pred vykonaním operácie withdraw musí byť suma výberu kladná a zároveň nesmie presiahnuť aktuálny zostatok na účte.

- **Odborne:**

Predpodmienka (precondition) definuje stav, ktorý musí byť splnený ešte pred vykonaním operácie. Ak nie je splnená, operácia sa nemá vykonať. self.balance reprezentuje stav objektu pred vykonaním operácie.

post – podmienka po vykonaní operácie

- **Príklad postpodmienka**

```
context Account::withdraw(amount: Real)
post: self.balance = self.balance@pre - amount
```

- **Popis:**

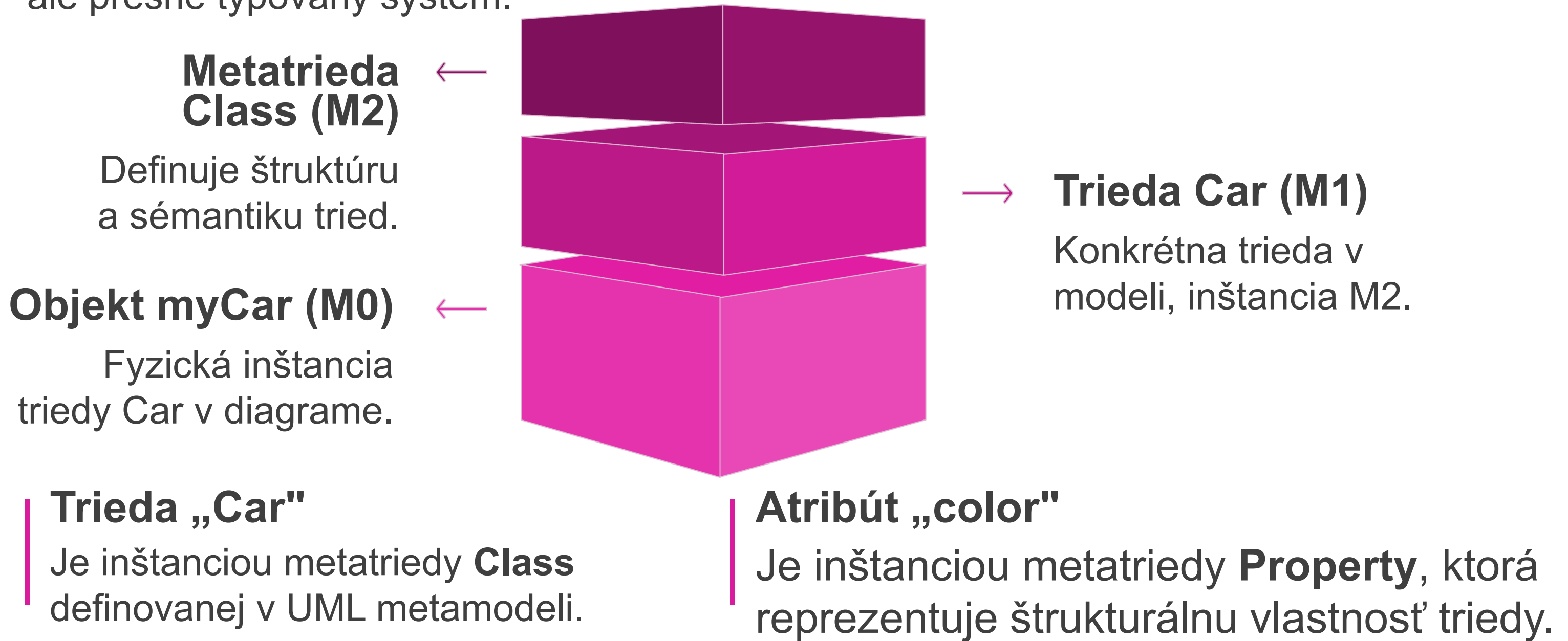
Po vykonaní operácie withdraw sa zostatok na účte zníži o vybranú sumu.

- **Odborne:**

Postpodmienka (postcondition) definuje očakávaný stav po vykonaní operácie. Výraz self.balance@pre označuje hodnotu atribútu pred vykonaním operácie, čo je štandardný mechanizmus OCL na porovnanie stavov.

Inštanciovanie v UML

Každý prvok UML modelu **je inštanciou metamodelu**. UML model nie je voľná kresba, ale presne typovaný systém.



UML CASE Nástroje

CASE nástroje využívajú metamodel na automatizované spracovanie modelov. Bez metamodelu by tieto nástroje nevedeli pracovať automatizovane.

1

Validácia

Kontrola konzistencie a správnosti modelu

2

Generovanie kódu

Automatická produkcia zdrojového kódu z modelu

3

Transformácie

Prevod medzi rôznymi typmi modelov

Príklady nástrojov: Enterprise Architect, MagicDraw / Cameo, Visual Paradigm



UML v Model-Driven Engineering

MDE & MDA

1. UML je základom prístupov **MDE (Model Driven Engineering)** a **MDA (Model Driven Architecture)**.
2. Model sa stáva primárnym artefaktom a kód je generovaný automaticky.

Transformačný reťazec

Metamodel zabezpečuje, že transformácie sú formálne korektné:

1. **PIM** → **PSM** (Platform Independent Model → Platform Specific Model)
2. **PSM** → **kód** (generovanie zdrojového kódu)

Výhody použitia metamodelu

Formálnosť a presnosť

Jazyk má jednoznačne definovanú syntax a sémantiku

Automatizácia a AI/ML

Umožňuje generovanie kódu a transformácie modelov

Interoperabilita

Nástroje rôznych výrobcov môžu spolupracovať

Validácia

Modely môžu byť automaticky overené voči pravidlám

✓ Z pohľadu praxe ide o kľúčový prvok profesionálneho modelovania.

Typické chyby a riziká

Zamieňanie diagramu za metamodel

Diagram je len vizuálna reprezentácia modelu, nie metamodel samotný.

Ignorovanie formálnej sémantiky

Používanie UML bez pochopenia presného významu prvkov vedie k nejednoznačným modelom.

Nesprávne vzťahy a nepochopenie vrstiev

Zámena asociácie, agregácie a kompozície, alebo nepochopenie vrstiev M0–M3, vedie k nevalidným modelom.

Príklady väzieb Model ↔ Metamodel

Nasledujúce príklady detailne vysvetľujú väzbu medzi modelom (M1) a metamodelom (M2) podľa OMG UML 2.5.1.

Vo všetkých prípadoch platí: **prvky UML diagramu sú inštancie metatried** definovaných v UML metamodeli.

01	02	03
Trieda a atribút	Asociácia	Dedičnosť
Class a Property	Association a Property	Generalization
04	05	
Operácia	Aktivita, Sekvencia, Constraint, Balík, Komponent, Rozhranie	
Operation a Parameter	Ďalšie metatriedy	

Príklad 1: Trieda a atribút

Model (M1)

Trieda **Student** s atribútom **name**:
String v diagrame tried.

Metamodel (M2)

- Trieda **Student** → inštancia metatriedy **Class**
- Atribút **name** → inštancia metatriedy **Property**
- **String** → inštancia **DataType**

Detailný popis

- Metatrieda **Class** obsahuje vzťah **ownedAttribute** - kolekciu objektov typu **Property**.
- Každý atribút má vlastnosti: **name**, **type**, **multiplicity**, **visibility**.
- Nástroje môžu validovať, či atribút má definovaný typ alebo či nedochádza k duplicitám.

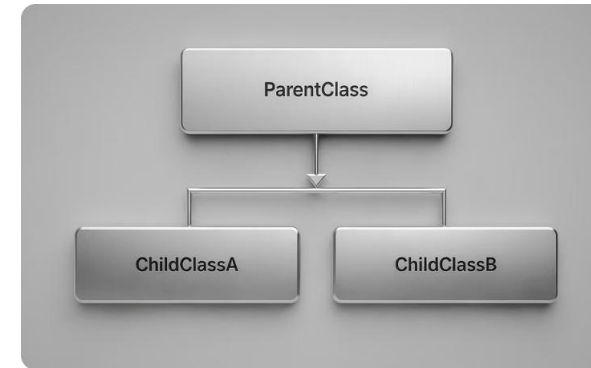
Príklad 2: Asociácia a Dedičnosť



Asociácia (Association)

Triedy **Student** a **Course** prepojené asociáciou „enrolls“.

Asociácia je inštancia metatriedy **Association**; každý koniec je **Property** v kolekcii **memberEnd** s multiplicitu, navigovateľnosťou a typom (napr. **0..***).



Dedičnosť (Generalization)

Trieda **Student** dedí z triedy **Person**. Vzťah je inštancia metatriedy **Generalization**; vlastnosť **specific** odkazuje na **Student**, **general** na **Person**.

Atribút **isSubstitutable** určuje, či môže byť potomok použitý namiesto rodiča.

Príklad 3: Operácia a parametre

Model (M1)

Trieda **Account** obsahuje operáciu:

```
withdraw(amount: Double): Boolean
```

Metamodel (M2)

- **withdraw** → inštancia **Operation**
- **amount** → inštancia **Parameter** (direction: in)
- návratová hodnota → **Parameter** (direction: return)

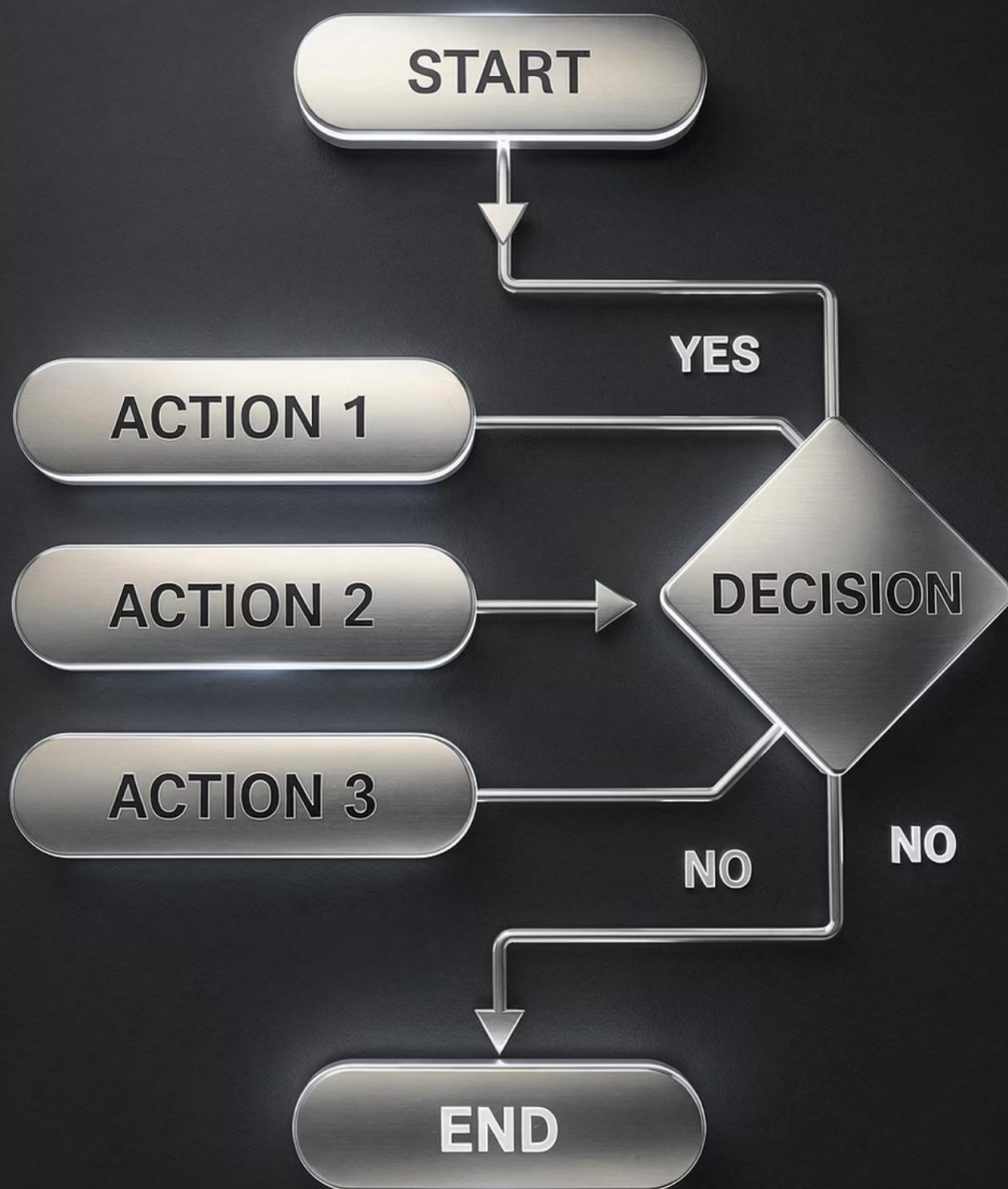
Detailný popis

Metatrieda **Operation** obsahuje kolekciu **ownedParameter**, kde každý parameter má vlastnosti **name**, **type**, **direction** (in, out, return). Typ **Double** je **DataType**.

V praxi to znamená, že UML model môže byť použitý ako základ pre **generovanie API** alebo kontrolu compatibility rozhraní.

Príklad 4: Aktivita

Model: Proces prihlasovania — Start → Login → Validate → End.



Activity

Celý diagram je inštancia metatriedy **Activity** (špecializácia **Behavior**)

ActivityNode

Kroky sú inštancie uzlov: **InitialNode**, **Action**, **DecisionNode**, **FinalNode**

ActivityEdge

Prechody sú **ControlFlow** alebo **ObjectFlow** s guard podmienkami a tokom dát

Príklad 5: Sekvenčný diagram

Model (M1)

Objekt **User** posiela správu `login()` objektu **System**.

Metamodel (M2)

- Celý diagram → inštancia `Interaction`
- Objekty → `Lifeline`
- Správa → `Message`

Detailný popis

Metatrieda `Message` obsahuje odkazy na `sendEvent` a `receiveEvent`. `Lifeline` reprezentuje účasť objektu v interakcii.

Každá správa môže byť **synchronna** alebo **asynchronna** - definované atribútom `messageSort`.

Umožňuje presnú analýzu komunikácie medzi komponentmi.



Príklad 6: Constraint a OCL

Model (M1)

Trieda **Person** má pravidlo: vek musí byť väčší ako 0.

```
context Person  
inv: self.age > 0
```

Metamodel (M2)

Constraint je inštancia metatriedy Constraint. Výraz je **OpaqueExpression** obsahujúci OCL. Vlastnosť **context** určuje, na ktorý prvok sa vzťahuje.

OCL výraz je formálny a jednoznačný - umožňuje **automatickú validáciu** modelu a zabraňuje neplatným hodnotám.

Príklad 7: Balík a Namespace

Model (M1)

Balík **SchoolSystem** obsahuje triedy **Student** a **Course**.

Metamodel (M2)

SchoolSystem je inštancia metatriedy **Package**, ktorá dedí z **Namespace**. Vlastnosť **ownedMember** obsahuje všetky prvky v balíku.

Detailný popis

Package organizuje prvky modelu a zabezpečuje menný priestor. Metamodel umožňuje aj **vnorené balíky**, čo je dôležité pri modelovaní rozsiahlych systémov.

Tento princíp je analogický k balíkom v programovacích jazykoch ako Java alebo C#.

Príklad 8: Komponent a Rozhranie



Component - PaymentService

- Inštancia metatriedy **Component**, ktorá dedí z **Class**.
- Metamodel definuje vzťahy **provided** a **required** pre modelovanie závislostí.
- Používa sa pri návrhu architektúry a nasadení systémov.



Interface - PaymentMethod

- Inštancia metatriedy **Interface**, ktorá dedí z **Classifier**.
- Definuje kontrakt bez implementácie.
- Realizácia je modelovaná vzťahom **Realization** - samostatnou metatriedou.
- Priamo využiteľné pri návrhu API.

Zhrnutie Metamodel

Vo všetkých príkladoch platí rovnaký princíp:

Prvky UML diagramu sú inštancie metatríd definovaných v UML metamodeli.

Formálny základ

Metamodel definuje štruktúru, prvky a pravidlá jazyka UML

Viac než vizualizácia

UML je formálny modelovací systém vhodný pre automatizáciu a validáciu

Profesionálne inžinierstvo

Bez metamodelu by UML stratilo presnosť a použiteľnosť v softvérovom inžinierstve



Referencie a zdroje

1

OMG UML 2.5.1

UML Version 2.5.1 Specification - Object Management Group

2

OMG MOF

Meta Object Facility (MOF) Core Specification - Object Management Group

3

Booch, Rumbaugh, Jacobson

The Unified Modeling Language User Guide

4

Fowler, M.

UML Distilled



Čo sa naučíme?

1. Čo je UML metamodel?
2. Aký je rozdiel medzi UML modelom a UML metamodelom?
3. Čo predstavujú vrstvy M0 až M3 v UML?
4. Čo je MOF (Meta Object Facility) a aký má vzťah k UML?
5. Čo je metatrieda v UML metamodeli?
6. Ako je v metamodeli definovaná trieda (Class)?
7. Ako metamodel reprezentuje vzťahy medzi prvkami?
8. Čo je Property v UML metamodeli?
9. Akú úlohu má OCL v UML metamodeli?
10. Prečo je UML metamodel dôležitý pre nástroje a automatizáciu?

Ako dostat' do EA UML Metamodel?



ISSUES ASSOCIATED WITH THIS SPECIFICATION



Issues Reported in this Specification - Version 2.5.1 only



Issues Fixed in this Specification - Version 2.5.1 only



Report an issue



Issue Tracker (Members Only)

SPECIFICATION DOCUMENTS

NORMATIVE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
Specification	PDF	UML/2.5.1/PDF	formal/17-12-05

NORMATIVE MACHINE READABLE DOCUMENTS

DESCRIPTION	FORMAT	URL	OMG FILE ID
UML 2.5.1 Abstract Syntax Metamodel	XMI	UML/20161101/UML.xmi	ptc/18-01-01
UML 2.5.1 Primitive Types	XMI	UML/20161101/PrimitiveTypes.xmi	ptc/18-01-02
UML 2.5.1 Standard Profile	XMI	UML/20161101/StandardProfile.xmi	ptc/18-01-03
UML 2.5.1 Diagram Interchange Metamodel	XMI	UML/20161101/UMLDI.xmi	ptc/18-01-04

INFORMATIVE DOCUMENTS





OMG Issue

UMLR — Need packages overview diagram and explicit depiction of package dependencies

Key: [UMLR-472](#) · OMG Task Force: [UML 2.6 RTF](#)

Open

Closed

All

Key: UMLR-472

Legacy Issue Number: 18195

Status: **OPEN**

Source: Simula Research Laboratory ([Dr. Bran Selic](#))

Summary: In the current 2.5 metamodel, there is a diagram that simply shows all the various packages that comprise it. Note, however, that we do not have such a diagram in the spec itself. In fact, as far as I can tell, it seems that the spec does not even mention explicitly that the metamodel is divided up into these packages.

Furthermore, it is useful to see the dependencies between the various packages captured explicitly in diagrams. In a sense, this shows the intended couplings between the various modules, which is usually important architectural information. Not showing them explicitly either in the spec or in a metamodel diagram is obscuring this and could lead to corruption of the architecture in subsequent maintenance (e.g., the introduction of undesired and even incorrect couplings between the packages).

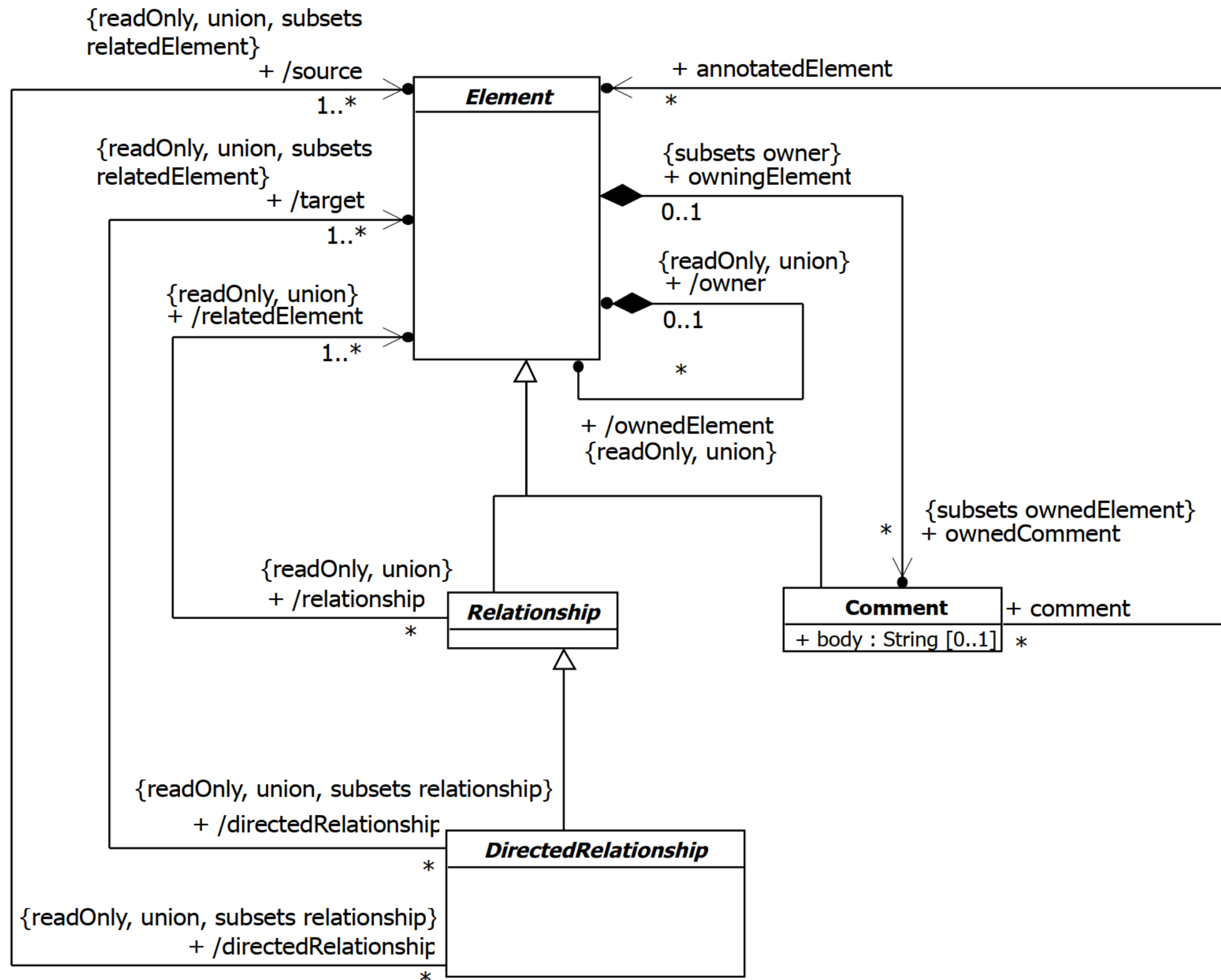
Reported: [UML 2.5b1](#) — Tue, 23 Oct 2012 04:00 GMT

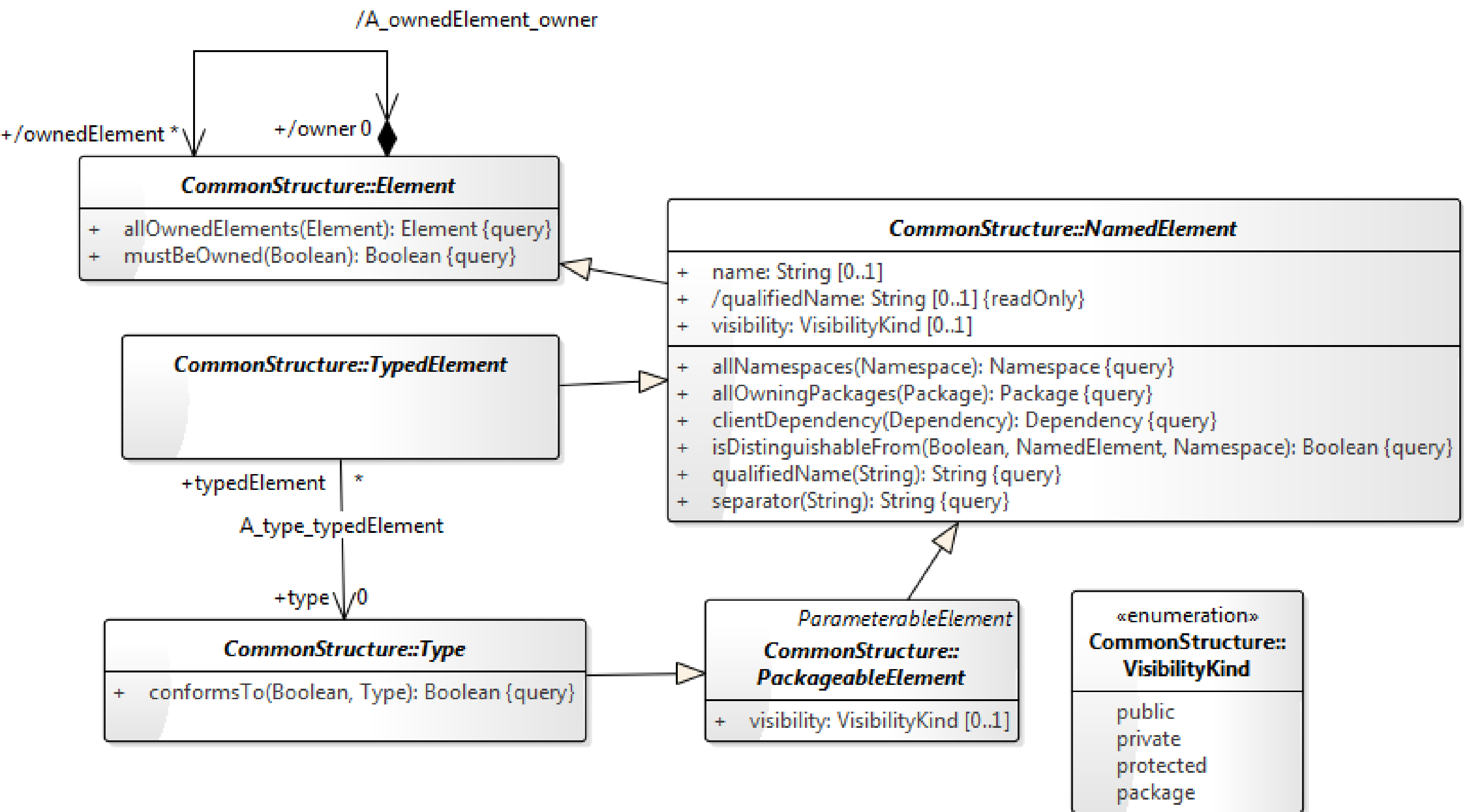
Updated: Fri, 6 Mar 2015 20:57 GMT

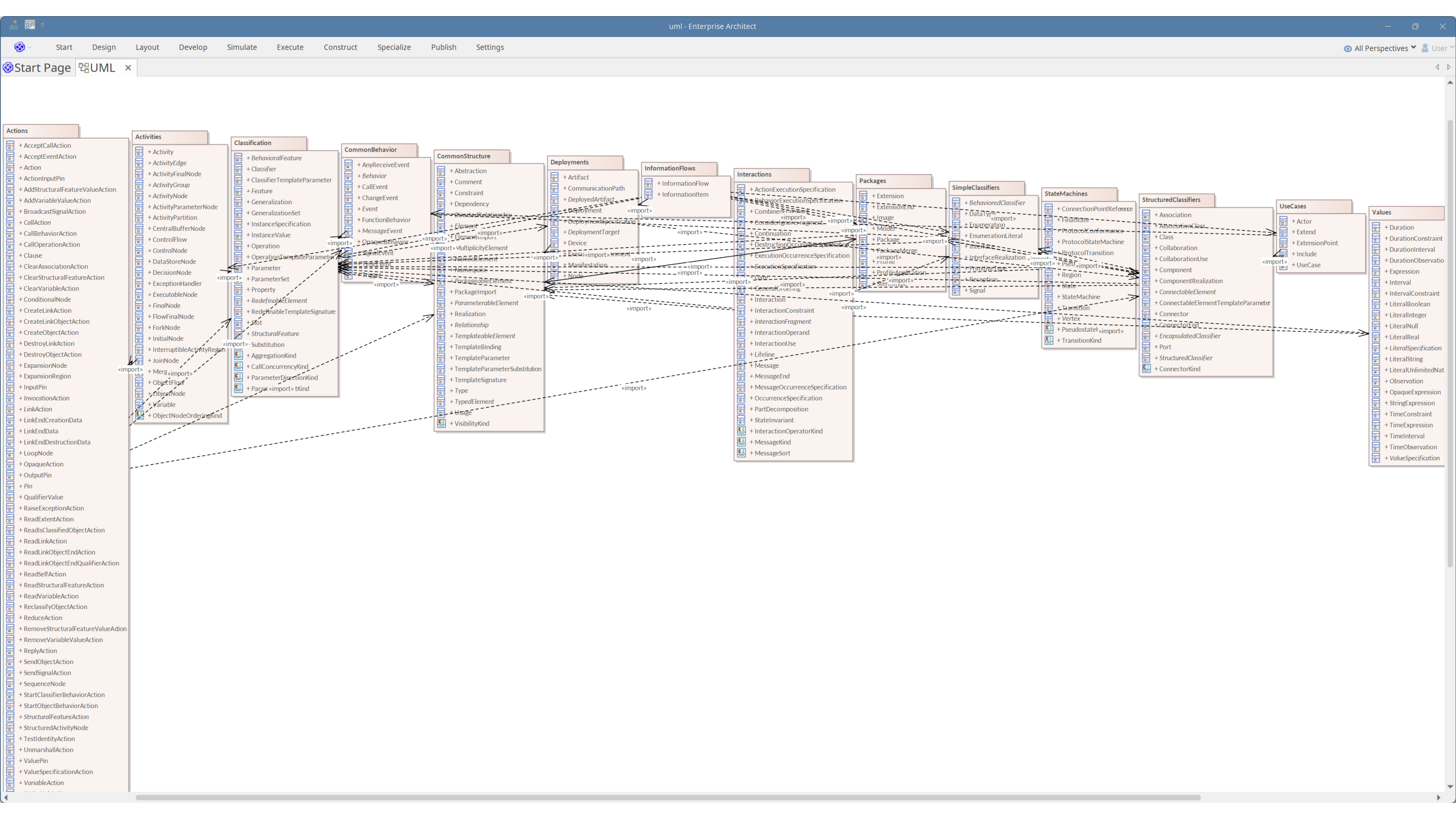


Diagram metamodelu UML

7.1 Root kapitole 7.2.2 Abstract Syntax







umt - Enterprise Architect

StartDesignLayoutDevelopSimulateExecuteConstructSpecializePublishSettings

Report BuilderCustom Document Model DocumentsTemplate EditorDynamic Preview

HTMLGenerate HelpPrintSaveReusable AssetsPublish As...CSVPackage ControlExportImport

Model ReportsDiagram ImageModel Exchange

Find Package

UML Class Diagram

Start PageUML

Model

- Imported UML 2.5.1.
 - UML

Actions

- + AcceptCallAction
- + AcceptEventAction
- + Action
- + ActionInputPin
- + AddStructuralFeatureValueAction
- + AddVariableValueAction
- + BroadcastSignalAction
- + CallAction
- + CallBehaviorAction
- + CallOperationAction
- + Clause
- + ClearAssociationAction
- + ClearStructuralFeatureAction
- + ClearVariableAction
- + ConditionalNode
- + CreateLinkAction
- + CreateLinkObjectAction
- + CreateObjectAction
- + DestroyLinkAction
- + DestroyObjectAction
- + ExpansionNode
- + ExpansionRegion
- + InputPin
- + InvocationAction
- + LinkAction
- + LinkEndCreationData
- + LinkEndData
- + LinkEndDestructionData
- + LoopNode
- + OpaqueAction
- + OutputPin
- + Pin
- + QualifierValue
- + RaiseExceptionAction
- + ReadExtentAction
- + ReadIsClassifiedObjectAction
- + ReadLinkAction
- + ReadLinkObjectEndAction
- + ReadLinkObjectEndQualifierAction
- + ReadSelfAction
- + ReadStructuralFeatureAction
- + ReadVariableAction
- + ReclassifyObjectAction
- + ReduceAction
- + RemoveStructuralFeatureValueAction
- + RemoveVariableValueAction
- + ReplyAction
- + SendSignalAction

Activities

- + Activity
- + ActivityEdge
- + ActivityFinalNode
- + ActivityGroup
- + ActivityNode
- + ActivityPartition
- + CentralBufferNode
- + ControlFlow
- + ControlNode
- + DataStoreNode
- + DecisionNode
- + ExceptionHandler
- + ExecutableNode
- + ForkNode
- + FinalNode
- + FlowFinalNode
- + ForkNode
- + InitialNode
- + InterruptibleActivityRegion
- + JoinNode
- + Merger
- + ObjectFlow
- + ObjectNode
- + Variable
- + ObjectNodeOrderingKind

Classification

- + BehavioralFeature
- + Classifier
- + ClassifierTemplateParameter
- + Feature
- + Generalization
- + GeneralizationSet
- + InstanceSpecification
- + InstanceValue
- + Operation
- + OperationTemplateParameter
- + Parameter
- + ParameterSet
- + Property
- + RedefinableElement
- + RedefinableTemplateSignature
- + Slot
- + StructuralFeature
- + Substitution
- + AggregationKind
- + CallConcurrencyKind
- + ParameterDirectionKind
- + ParameterKind

Common

- + Ar
- + Be
- + Ca
- + Ch
- + Fu
- + M
- + S
- + St
- + T
- + U

Interactions

- + ActionExecutionSpecification
- + BehaviorExecutionSpecification
- + CombinedParameter
- + Consideration
- + Continuation
- + DestructionOccurrenceSpecification
- + ExecutionOccurrenceSpecification
- + ExecutionSpecification
- + Generalization
- + Interaction
- + InteractionConstraint
- + InteractionFragment
- + InteractionOperand
- + InteractionUse
- + Lifeline
- + Message
- + MessageEnd
- + MessageOccurrenceSpecification
- + OccurrenceSpecification
- + PartDecomposition
- + StateInvariant
- + InteractionOperatorKind
- + MessageKind
- + MessageSort

Packages

- + Extension
- + ExtensionEnd
- + Image
- + Model
- + Package
- + PackageMerge
- + ProfileApplication
- + Profile

SimpleClassifiers

- + BehavedClassifier
- + DataType
- + Enumeration
- + EnumerationLiteral
- + InterfaceRealization
- + PrimitiveType
- + Reception
- + Signal

StateMachines

- + ConnectionPointReference
- + FinalState
- + ProtocolConformance
- + ProtocolStateMachine
- + ProtocolTransition
- + Pseudostate
- + Region
- + State
- + StateMachine
- + Transition
- + Vertex
- + Pseudostate
- + TransitionKind

Import Package from Native/XML File

PackageUML

Filename\\miros\Desktop\UML 2.5.1 Diagram Interchange Metamodel.xmlBrowse

Import Options

☒ Import Diagrams☐ Strip GUIDs

☐ Import using Single Transaction (Not recommended for large imports)☒ Write Log File

☒ Check if Package can be imported as Model Root

Treat Imported Datatypes as :

Merge Options

Merge XML☐ Baseline Package after Import

View XMLImportCloseHelp

Progress

Properties

General

NameUML

TypePackage

Stereotype

Alias

Keywords

StatusProposed

Version1.0

Package

URI

VisibilityPublic

Project

Notes

B I U

Class Diagram:UML: created: 23. 4. 2026 11:35:35 modified: 23. 4. 2026 11:37:43 88% 803 x 1146

All Perspectives